

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

I Mục lục

1. Bảng thuật ngữ 4

1.

CHƯƠNG 1.

Bảng thuật ngữ

Thuật ngữ kỹ thuật sử dụng trong sách, sắp xếp theo bảng chữ cái. Phụ lục này đồng bộ với bảng thuật ngữ chuẩn của sách.

English	Giải thích ngắn	Chương đầu
Actor / Subject / Delegation	Trong authorization: subject là user được đại diện; actor là workload đang gọi; delegation là quyền actor hành động thay subject. Ba phần phải tách biệt để tránh confused deputy.	Ch.9
Aggregate	Cụm entity/value object được xem như một đơn vị nhất quán, truy cập qua một “gốc” (aggregate root) để bảo vệ ràng buộc nghiệp vụ.	Ch.2
Anti-Corruption Layer	ACL — lớp dịch trung gian giúp một service không bị “nhiễm” mô hình dữ liệu của hệ thống ngoài/legacy khi tích hợp.	Ch.10
API Composition	Gộp dữ liệu từ nhiều service bằng cách gọi lần lượt rồi ghép kết quả ở tầng gọi (thay cho join trong database).	Ch.7
API Gateway	Lớp xử lý request tại biên, tập trung các trách nhiệm cross-cutting như routing, authentication và rate limiting; không sở hữu business authorization của từng service.	Ch.8
Backend for Frontend	BFF — một gateway riêng “may đo” cho từng loại client (web, mobile), trả về dữ liệu vừa vận nhu cầu client đó.	Ch.8

English	Giải thích ngắn	Chương đầu
BASE	Basically Available, Soft state, Eventually consistent — triết lý dữ liệu phân tán đối trọng với ACID: ưu tiên tính sẵn sàng, chấp nhận trạng thái trung gian và nhất quán cuối cùng.	Ch.7
Blue/Green Deployment	Chạy song song hai môi trường (cũ “blue” và mới “green”), chuyển traffic sang bản mới khi sẵn sàng và có thể chuyển lại nếu phiên bản dữ liệu/API vẫn tương thích.	Ch.12
Bottleneck	Điểm nghẽn — thành phần chậm nhất giới hạn hiệu năng của toàn hệ thống.	Ch.1
Bounded Context	Phạm vi mà một mô hình domain, bộ thuật ngữ và quy tắc có hiệu lực. Đây là ranh giới mô hình; không mặc nhiên ánh xạ một-một sang deployable service.	Ch.2
Build	Biên dịch mã nguồn và đóng gói thành artifact chạy được.	Ch.12
Bulkhead	Cô lập tài nguyên (thread pool, connection) theo nhóm để một phần quá tải không gây đình trệ phần còn lại. Ẩn dụ: khoang kín của tàu thủy.	Ch.4
Business Logic	Phần mã nguồn cốt lõi thực thi các quy tắc, tính toán và luồng nghiệp vụ đặc thù của miền.	Ch.1
Canary Deployment	Phát hành dần cho một nhóm nhỏ người dùng trước, theo dõi, rồi mới mở rộng toàn bộ.	Ch.12
CAP Theorem	Định lý CAP — khi mạng bị phân vùng (Partition), hệ phân tán buộc phải chọn giữa Consistency (nhất quán) và Availability (sẵn sàng); không thể có trọn vẹn cả ba.	Ch.7
Change Data Capture	CDC — bắt các thay đổi trong database (insert/update/delete) thành luồng sự kiện cho hệ thống khác tiêu thụ.	Ch.7
Chaos Engineering	Chủ động gây lỗi có kiểm soát trên hệ thống thật để kiểm chứng khả năng chịu lỗi.	Ch.11
Choreography	Cách phối hợp saga không có “nhạc trưởng”: mỗi service tự phản ứng với event và phát event tiếp theo.	Ch.6
CI/CD	Continuous Integration / Continuous Delivery — tự động build, test, triển khai mỗi khi mã nguồn thay đổi.	Ch.12
Circuit Breaker	“Ngắt mạch” tạm dừng gọi tới một service đang lỗi để tránh lỗi lan và cho nó thời gian hồi phục. Ẩn dụ: cầu dao điện.	Ch.4

English	Giải thích ngắn	Chương đầu
Cluster	Nhóm nhiều máy/node hoạt động như một hệ thống thống nhất.	Ch.12
Compensating Transaction	Giao dịch bù trừ — thao tác “làm ngược lại” để hoàn tác hệ quả của một bước đã thực hiện trong saga (vì không thể rollback như transaction database).	Ch.6
Consumer-Driven Contract Testing	CDCT — consumer công bố kỳ vọng về request/response, provider xác minh contract đó trong CI trước khi deploy (ví dụ: Pact). Sách dành CDC cho Change Data Capture.	Ch.3
Container	Gói đóng kín ứng dụng cùng mọi phụ thuộc để chạy giống nhau ở mọi môi trường.	Ch.12
Contract Testing	Kiểm thử xác nhận hai service tuân thủ đúng “hợp đồng” API đã thỏa thuận, không cần chạy cả hai cùng lúc.	Ch.3
Conway’s Law	Cấu trúc hệ thống có xu hướng phản chiếu cấu trúc giao tiếp của tổ chức tạo ra nó.	Ch.2
Core Domain	Subdomain tạo khác biệt chiến lược và đáng nhận đầu tư thiết kế cao nhất; không phải tên gọi chung cho mọi Bounded Context nghiệp vụ.	Ch.2
Correlation ID	Mã định danh cho một workflow nghiệp vụ, có thể liên kết nhiều request, trace và message; không đồng nghĩa với request ID hoặc trace ID.	Ch.8
CQRS	Command Query Responsibility Segregation — tách mô hình ghi (command) và mô hình đọc (query) thành hai đường riêng để tối ưu độc lập.	Ch.7
Cross-cutting concerns	Các nghiệp vụ cắt ngang toàn bộ hệ thống (như bảo mật, logging, rate limiting) không thuộc về một domain cụ thể nào.	Ch.8
DB Queue	Dùng một bảng database làm hàng đợi công việc — đơn giản hơn message broker cho nhu cầu nhỏ.	Ch.5
Dead Letter Topic/Queue	DLT — nơi chứa message xử lý thất bại sau nhiều lần thử, để xử lý riêng thay vì retry vô hạn.	Ch.5
Debug	Tìm và sửa nguyên nhân lỗi trong phần mềm.	Ch.1
Distributed Monolith	Phản-pattern: đã có service boundaries hình thức nhưng data, library hoặc deployment vẫn coupling theo lockstep, nên các service không thể tiến hóa độc lập.	Ch.1

English	Giải thích ngắn	Chương đầu
Distributed Tracing	Theo dõi một request xuyên qua nhiều service bằng trace/span có đo thời gian từng bước – trả lời câu hỏi “chậm ở đâu” trong hệ phân tán.	Ch.11
Docker Compose	Công cụ định nghĩa và chạy nhiều container cùng lúc bằng một file YAML.	Ch.12
Domain-Driven Design	DDD – phương pháp thiết kế lấy mô hình nghiệp vụ (domain) làm trung tâm, dùng ngôn ngữ chung giữa lập trình viên (developer) và chuyên gia nghiệp vụ.	Ch.2
Downtime	Khoảng thời gian hệ thống không phục vụ được.	Ch.4
DTO	Data Transfer Object – đối tượng chỉ để truyền dữ liệu qua ranh giới (API), tách biệt với entity nội bộ.	Ch.3
Effectively-once	Đảm bảo “như thể xử lý đúng một lần” bằng at-least-once + idempotency (vì exactly-once tuyệt đối xuyên hệ là bất khả thi).	Ch.5
Endpoint	Điểm cuối giao tiếp mạng (thường là một URL cụ thể) nơi API tiếp nhận và trả về dữ liệu cho client.	Ch.3
Event Sourcing	Lưu trạng thái dưới dạng chuỗi sự kiện bất biến (thay vì chỉ trạng thái hiện tại); trạng thái được tái dựng bằng cách “phát lại” sự kiện. Ấn dụ: sổ sao kê ngân hàng.	Ch.7
Event Storming	Workshop dùng sticky note để cùng khám phá domain qua các sự kiện nghiệp vụ.	Ch.2
Event-driven	Hướng sự kiện – kiến trúc trong đó các service giao tiếp gián tiếp bằng cách phát và nghe sự kiện.	Ch.5
Eventual Consistency	Dữ liệu giữa các service cuối cùng sẽ đồng bộ, nhưng có một khoảng trễ được chấp nhận – không có cận trên cam kết (đối lập với strong consistency).	Ch.6
Flyway / Liquibase	Công cụ quản lý phiên bản schema database (versioned migrations), chạy như một bước riêng trước khi deploy service.	Ch.12
GitOps	Quản lý hạ tầng/triển khai bằng Git làm nguồn sự thật duy nhất; mọi thay đổi qua commit/PR.	Ch.12
GraphQL	Ngôn ngữ truy vấn API cho phép client yêu cầu đúng dữ liệu cần, tránh over/under-fetching.	Ch.3
gRPC	Framework RPC hiệu năng cao của Google chạy trên HTTP/2 + Protobuf, hợp cho giao tiếp nội bộ service-to-service.	Ch.4

English	Giải thích ngắn	Chương đầu
HATEOAS	Mức “trưởng thành” của REST trong đó response kèm các link điều hướng hành động tiếp theo.	Ch.3
Header	Tiêu đề của một gói tin hoặc yêu cầu HTTP, dùng để truyền tải các siêu dữ liệu (metadata) như token xác thực.	Ch.8
Hexagonal Architecture	Ports & Adapters — tách lỗi nghiệp vụ khỏi hạ tầng qua “cổng” và “bộ chuyển đổi”, giúp dễ test và thay công nghệ.	Ch.3
Idempotency (tính lũy đẳng)	Tính chất của một thao tác: thực hiện nhiều lần cho kết quả giống như thực hiện một lần duy nhất. Ấn dụ: bấm nút gọi thang máy 5 lần, thang vẫn chỉ đến một lần.	Ch.4
Independent Silo Monoliths	Nhiều ứng dụng monolith tách rời, mỗi ứng dụng có codebase/deployment riêng nhưng thiếu kiến trúc và contract thống nhất; khác Distributed Monolith.	Ch.1
Infrastructure as Code	IaC — mô tả hạ tầng bằng mã/khai báo để tạo lại tự động và nhất quán.	Ch.12
JWKS	JSON Web Key Set — tập public keys do issuer công bố để resource server xác minh token và tự làm mới khóa khi rotation.	Ch.9
JWT	JSON Web Token — định dạng token; sách chủ yếu dùng compact JWS có chữ ký để resource server verify claims mà không cần tra database ở mỗi request.	Ch.9
k3s	Bản Kubernetes gọn nhẹ, hợp cho edge và môi trường tài nguyên thấp.	Ch.12
Load Balancing	Phân phối request đến nhiều instance để tránh quá tải một instance.	Ch.4
Message broker	Trung gian nhận/lưu/chuyển message giữa producer và consumer, cho phép giao tiếp bất đồng bộ. Ấn dụ: bưu điện giữ thư hộ.	Ch.5
Microservice	Dịch vụ nhỏ, tự chủ, triển khai độc lập, sở hữu dữ liệu riêng và giao tiếp với nhau qua mạng.	Ch.1
Modular Monolith	Ứng dụng đơn khối nhưng chia module ranh giới rõ ràng bên trong — một lựa chọn hợp lệ trước khi tách microservices.	Ch.1
Monolith	Kiến trúc nguyên khối: ứng dụng được đóng gói và triển khai như một khối duy nhất.	Ch.1

English	Giải thích ngắn	Chương đầu
mTLS	Mutual TLS — hai phía xác thực certificate/workload identity và thiết lập kênh mã hóa. mTLS không tự chứng minh nội dung user claims hoặc cấp business authorization.	Ch.9
OAuth2	Chuẩn ủy quyền cho phép cấp quyền truy cập tài nguyên mà không chia sẻ mật khẩu.	Ch.9
Observability	Khả năng hiểu trạng thái bên trong hệ thống từ dữ liệu nó phát ra (logs, metrics, traces) — đủ để đặt câu hỏi mới mà không cần triển khai (deploy) lại.	Ch.11
OIDC	OpenID Connect — lớp xác thực (authentication) xây trên OAuth2, bổ sung danh tính người dùng qua ID token.	Ch.9
OpenAPI	OpenAPI / Swagger — chuẩn mô tả REST API dạng máy đọc được, dùng để sinh tài liệu và code.	Ch.3
OpenFeign	Thư viện Spring cho phép gọi REST service khác qua khai báo interface (declarative HTTP client).	Ch.4
Orchestration	Cách phối hợp saga có “nhạc trưởng” trung tâm điều khiển từng bước.	Ch.6
Outbox Pattern	Ghi sự kiện vào một bảng “outbox” cùng transaction với dữ liệu nghiệp vụ, rồi publish riêng — tránh mất/đôi sự kiện do dual-write.	Ch.10
Overhead	Chi phí tài nguyên/độ trễ phát sinh thêm ngoài công việc chính.	Ch.1
Pact	Công cụ contract testing theo hướng consumer-driven.	Ch.3
Pipeline	Chuỗi bước tự động (build, test, deploy...) nối tiếp nhau.	Ch.12
PKCE	Proof Key for Code Exchange — cơ chế bảo vệ luồng OAuth2 Authorization Code cho client công khai (mobile/SPA) chống đánh cắp mã.	Ch.9
Poison Pill	Message lỗi/không xử lý được làm consumer kẹt vòng lặp retry; cần tách sang DLT.	Ch.5
Polyglot Persistence	Mỗi service tự chọn loại database phù hợp nhất với nhu cầu của nó (SQL, NoSQL, cache...).	Ch.7
Rate Limiting	Giới hạn số request trong một khoảng thời gian để bảo vệ hệ thống khỏi quá tải/lạm dụng.	Ch.8
RBAC	Role-Based Access Control — phân quyền theo vai trò được gán cho người dùng.	Ch.9

English	Giải thích ngắn	Chương đầu
Refactor	Cải thiện cấu trúc mã mà không thay đổi hành vi.	Ch.1
Resilience4j	Thư viện Java hiện thực các resilience pattern (Circuit Breaker, Retry, Timeout, Bulkhead) qua annotation/config.	Ch.4
Resource Server	Service/API sở hữu tài nguyên được bảo vệ; nó xác minh access token dành cho mình và thực hiện local authorization.	Ch.9
REST	Kiểu kiến trúc API dựa trên resource và HTTP verbs (GET/POST/PUT/DELETE).	Ch.3
Retry	Thử lại request khi gặp lỗi tạm thời.	Ch.4
Routing	Quá trình định tuyến, điều hướng request từ nguồn đến đúng đích dựa trên các quy tắc (path, header).	Ch.8
Saga pattern	Phối hợp chuỗi local transaction; khi bước sau thất bại, compensating action đưa nghiệp vụ về trạng thái chấp nhận được, không nhất thiết khôi phục bit-for-bit như rollback database.	Ch.6
Scaling	Mở rộng khả năng phục vụ bằng cách thêm tài nguyên (scale up) hoặc thêm instance (scale out).	Ch.1
Service	Một thành phần phần mềm tự chủ cung cấp một nhóm chức năng qua giao diện rõ ràng.	Ch.1
Service Discovery	Cơ chế để service tìm địa chỉ động của service khác mà không hard-code IP.	Ch.4
Service Mesh	Lớp hạ tầng (thường qua sidecar) lo giao tiếp service-to-service: routing, mTLS, retry, observability.	Ch.12
Service Registry	Danh bạ trung tâm lưu địa chỉ các instance đang hoạt động; service tự đăng ký (register) và bên gọi tra cứu để định tuyến (ví dụ: Netflix Eureka).	Ch.4
Sidecar	Sidecar pattern — đặt một container phụ chạy cạnh service chính để lo các việc chung (proxy, logging) mà không phải sửa đổi mã nguồn của dịch vụ.	Ch.12
Signed Identity Propagation	Truyền token/metadata có chữ ký qua trust boundary; service đích kiểm tra issuer, audience, signature và freshness trước khi dùng claims.	Ch.8
SLI/SLO/SLA	Chỉ số đo chất lượng dịch vụ (SLI), mục tiêu nội bộ (SLO) và cam kết với khách hàng (SLA).	Ch.11

English	Giải thích ngắn	Chương đầu
Strangler Fig	Strangler Fig pattern — thay thế dần hệ cũ bằng cách bọc và chuyển từng phần chức năng sang hệ mới, đến khi hệ thống cũ bị thay thế hoàn toàn.	Ch.10
Strong Consistency	Nhất quán mạnh — mọi read luôn thấy write mới nhất, không có khoảng tạm thời đọc phải dữ liệu cũ (stale).	Ch.7
Sysbox	Runtime container cho phép chạy Docker/Kubernetes lồng bên trong container an toàn hơn.	Ch.12
Team	Nhóm/đội phát triển.	Ch.1
Tiered Trust	Mô hình tăng bằng chứng theo rủi ro: T0 ingress chưa tin cậy; T1 signed user proof + local authorization; T2 thêm workload/actor/delegation; T3 bảo vệ job/command bất đồng bộ.	Ch.9
Timeout	Bên gọi ngừng chờ sau một thời hạn; công việc phía remote có thể vẫn tiếp tục, nên flow nhạy cảm cần cancellation, idempotency hoặc fencing phù hợp.	Ch.4
Traffic	Lưu lượng truy cập mạng hoặc khối lượng dữ liệu trao đổi giữa các thành phần hệ thống.	Ch.8
Turnstile	Cloudflare Turnstile — dịch vụ challenge chống bot (thay thế CAPTCHA), bảo vệ các form nhạy cảm như đăng nhập/đăng ký/khôi phục mật khẩu.	Ch.9
Two-Phase Commit	2PC — giao thức commit phân tán hai pha (prepare rồi commit); đảm bảo nhất quán nhưng khóa tài nguyên và kém chịu lỗi, ít dùng trong microservices.	Ch.6
WebSocket / SSE	Hai cơ chế server đẩy dữ liệu real-time về client: WebSocket (hai chiều, full-duplex) và Server-Sent Events (một chiều server→client).	Ch.5
Workload Identity	Danh tính mật mã của service/job đang chạy, dùng cho service-to-service authentication; khác với user subject mà workload có thể đại diện.	Ch.9
Zero Trust	Không cấp implicit trust chỉ vì vị trí mạng; mỗi lần truy cập resource cần authentication và authorization phù hợp với policy.	Ch.9

 **Lưu ý**

Lưu ý: Lần đầu xuất hiện trong text, thuật ngữ được in nghiêng kèm giải thích tiếng Việt. Sau đó dùng thuật ngữ tiếng Anh trực tiếp.